

学号： 1906410229

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 递归与分治

2022年秋季学期

上机时间：第 5 周 星期 五 第 3 ~ 4 节

学生姓名： 蔡艺洵 专业班级： 计算机 1902

指导教师： 刘俊岭 成 绩：

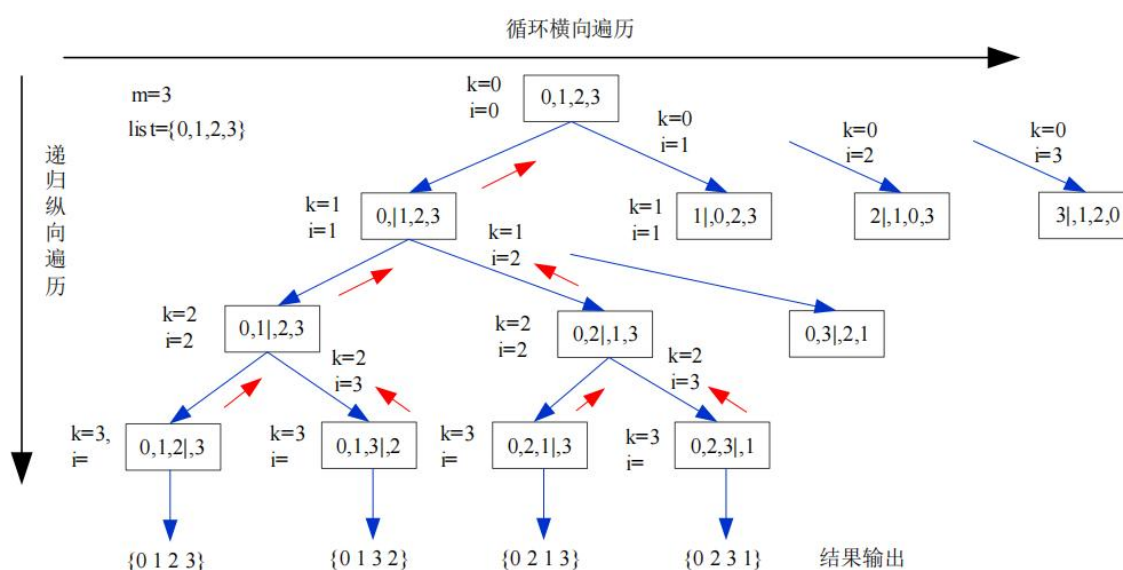
项 目 名 称	递 归 与 分 治	学 时	4
日期： 2022 年 9 月 30 日		第 5 教学周	
硬件及软件			
名 称		版本、型号	
Dev-C++		5.11	
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。 本次任务： 实验 2-1 全排列问题 实验 2-2 整数划分问题 实验 2-3 大整数乘法问题 实验 2-4 棋盘覆盖问题			

实验 2-1 全排列问题

1.问题描述:

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \dots, r_n\}$ 的全排列。设 $R = \{r_1, r_2, \dots, r_n\}$ 是要进行排列的 n 个元素， $R_i = R - \{r_i\}$ 。集合 X 中元素的全排列记为 $\text{perm}(X)$ 。 $(r_i)\text{perm}(X)$ 表示在全排列 $\text{perm}(X)$ 的每一个排列前加上前缀 r_i 得到的排列。

2.算法实例:



3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>

using namespace std;

void perm(int list[], int k, int m);
void swap(int& a, int& b);

int main()
{
```

```

int a[5] = {0,1,2,3,4};
perm(a, 0, 3);
printf("\n");

system("PAUSE");
return 0;
}
void perm(int list[], int k, int m)
{
    if (k == m)
    {
        //只剩下一个元素
        for (int i = 0; i <= m; i++)
            cout << list[i];
        cout << endl;
    }
    else
        for (int i = k; i <= m; i++)
        {
            swap(list[i], list[k]); //将第 i 个元素与第 k 个元素交换位置
            perm(list, k+1, m); //递归排列第 k+1 到第 m 个元素
            swap(list[i], list[k]); //第 i 和第 k 个交换元素的重新换回正常位置
        }
}
void swap(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

4.程序验证:

```

int a[5] = {0,1,2,3,4};
perm(a, 0, 3);
printf("\n");

```

验证组合 0, 1, 2, 3 的全排列, 输入为 0, 1, 2, 3, 输出结果为 0, 1, 2, 3 的全排列, 结果如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-1.exe

```
0123
0132
0213
0231
0321
0312
1023
1032
1203
1230
1320
1302
2103
2130
2013
2031
2301
2310
3120
3102
3210
3201
3021
3012
```

请按任意键继续. . .

验证组合 0, 1, 2 的全排列, 输入为 0, 1, 2 输出结果为 0, 1, 2 的全排列, 结果如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-1.exe

```
012
021
102
120
210
201
```

请按任意键继续. . .

5. 算法分析:

该算法使用了递归, 是递归算法。递归算法就是把问题分解成规模更小、具有与原来问题相同解法的问题。R 的全排列可归纳定义如下:

- 当 $n=1$ 时, $\text{perm}(R)=(r)$, 其中 r 是集合 R 中唯一的元素;
- 当 $n>1$ 时, $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$, $(r_2)\text{perm}(R_2)$, $\dots$, $(r_n)\text{perm}(R_n)$ 构成。

时间复杂度: $O(n \cdot n!)$

空间复杂度: $O(n)$

实验 2-2 整数划分问题

1.问题描述:

将正整数 n 表示成一系列正整数之和: $n=n_1+n_2+\cdots+n_k$, 其中 $n_1 \geq n_2 \geq \cdots \geq n_k \geq 1$, $k \geq 1$ 。正整数 n 的这种表示称为正整数 n 的划分。求正整数 n 的不同划分个数。

2.算法实例:

例如正整数6有如下11种不同的划分: 写出递归调用过程:

6;	$p(6)=q(6,6);$
5+1;	$q(6,5)+1(6);$
4+2, 4+1+1;	$q(6,4)+q(1,5)(5+1);$
3+3, 3+2+1, 3+1+1+1;	$q(6,3)+q(2,4);$
2+2+2, 2+2+1+1, 2+1+1+1+1;	$q(6,2)+q(3,3)+q(2,2)(4+2, 4+1+1)$
1+1+1+1+1+1。	$q(6,1)+q(4,2)+q(3,2)+1+q(2,2)$
	$1+1+1+1+1+1 \quad 3+2+1 \quad 3+1+1+1 \quad 3+3$
	$q(4,2)=q(4,1)+q(2,2) \quad q(2,1)+1$
	$2+1+1+1+1 \quad 2+2+1+1 \quad 2+2+2$

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n-1) & n = m \\ q(n, m-1) + q(n-m, m) & n > m > 1 \end{cases}$$

3.程序代码:

```
#include <math.h>
#include <tchar.h>
#include<iostream>
using namespace std;
```

```
int q(int n, int m) //最大加数 n1 不大于 m 的划分个数即为 q(n,m) m 为 n 的最大划分数
{
    if (n < 1 || m < 1) return 0; // n=0 或者 m=0 均不符合题意
    if (n == 1 || m == 1) return 1; // n=1 或者 m=1 都只有一种情况
    if (n < m) return q(n,n); // 因为 n>=m 为前提条件 所以这种情况 q(n,m)即为 q(n,n)
    if (n == m) return q(n,m-1) + 1; //n=m 的情况下, 即划分为 n1=n 和 n1<=n-1 的情况
    return q(n,m-1) + q(n-m,m); //正常情况下 q(n,m)的划分由 n1=m 的情况和 n1<=m-1 的情况
```

```

}

int main() {
    cout << q(6, 6) << endl; //输入值，可实际更改，如下的 6 7 8 5 4
    system("pause");
    return 0;
}

```

4.程序验证:

```

5 int main() {
6     cout << q(6, 6) << endl;
7     cout << q(7, 7) << endl;
8     cout << q(8, 8) << endl;
9     cout << q(5, 5) << endl;
0     cout << q(4, 4) << endl;
1     system("pause");
2     return 0;
3 }

```

分别验证整数 6, 7, 8, 5, 4 的划分，输入为 6, 7, 8, 5, 4 输出结果为 11, 15, 22, 7, 5，结果如图所示：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-2.exe

```

11
15
22
7
5
请按任意键继续. . .

```

5.算法分析:

设 $p(n)$ 为正整数 n 的划分数，则难以找到递归关系，因此考虑增加一个自变量：将最大加数 n_1 不大于 m 的划分个数记作 $q(n, m)$ 。

可以建立 $q(n, m)$ 的如下递归关系：

(1) $q(n, n) = 1 + q(n, n-1)$; 正整数 n 的划分由 $n_1 = n$ 的划分和 $n_1 \leq n-1$ 的划分组成。

(2) $q(n, m) = q(n, m-1) + q(n-m, m), n > m > 1$;

正整数 n 的最大加数 n_1 不大于 m 的划分由 $n_1 = m$ 的划分和 $n_1 \leq n-1$ 的划分组成。

时间复杂度: $O(n^2)$

空间复杂度: $O(n^2)$

实验 2-3 大整数乘法问题

1.问题描述:

分治法求两个 N 位的整数的乘积。

2.算法实例:

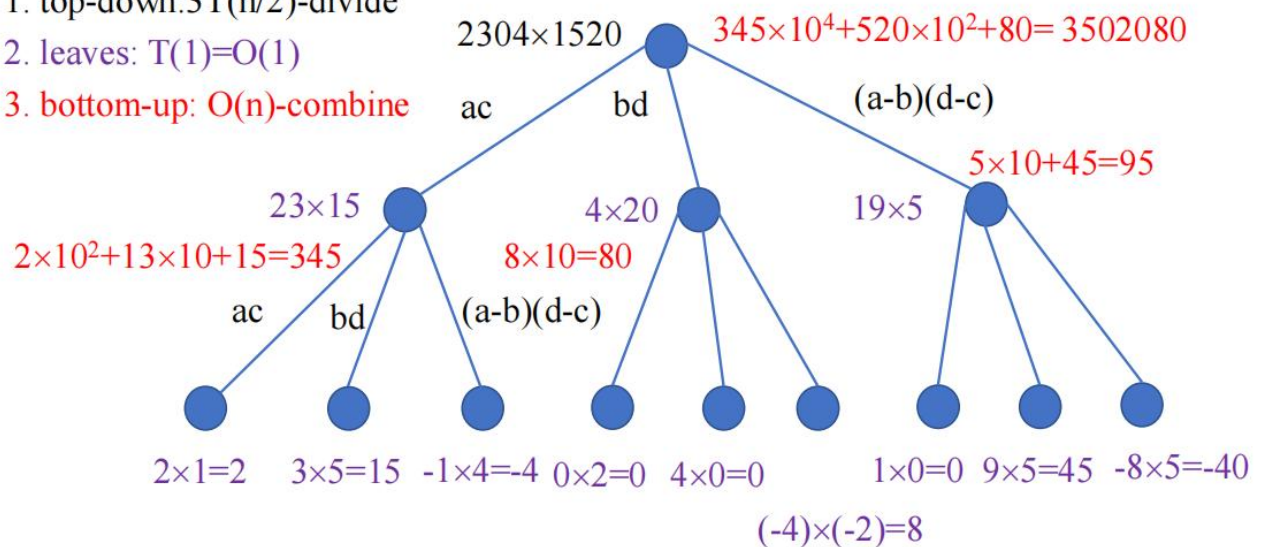
$$XY = ac \cdot 10^n + ((a-b)(d-c) + ac + bd) \cdot 10^{n/2} + bd$$

采用分治法计算 2304×1520 ，在递归树的结点处依次标注子问题及其解

1. top-down: $3T(n/2)$ -divide

2. leaves: $T(1)=O(1)$

3. bottom-up: $O(n)$ -combine



3.程序代码:

```
#include <math.h>
#include <tchar.h>
#include<iostream>
using namespace std;

int IntegerMultiply(int x, int y, int N)
{
    if ((0 == x) || (0 == y))
        return 0;
    if (1 == N)
        return x*y;
    else
    {
        int a = x / (int)pow(10., (int)N / 2);
        int b = x - a * (int)pow(10., N / 2);
```

```

        int c = y / (int)pow(10., (int)N / 2);
        int d = y - c * (int)pow(10., N / 2);

        int ac = IntegerMultiply(a,c,N / 2);
        int bd = IntegerMultiply(b,d,N / 2);
        int ambdmc = IntegerMultiply(a-b,d-c,N / 2) + ac + bd;
        return (ac * (int)pow(10., N) + ambdmc * (int)pow(10., N / 2) + bd);
    }
}

int _tmain(int argc, _TCHAR* argv[])
{
    int x = 2304;//输入值
    int y = 1520;//输入值

    cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
    cout << "x * y = " << x*y << endl;
    system("pause");
    return 0;
}

```

4.程序验证:

```

int x = 2304;//输入值
int y = 1520;//输入值
cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
cout << "x * y = " << x*y << endl;
x=1101;y=1010;
cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
cout << "x * y = " << x*y << endl;
x=1234;y=4321;
cout << "x * y = " << IntegerMultiply(x, y, 4) << endl;
cout << "x * y = " << x*y << endl;
system("pause");
return 0;

```

分别验证整数 2304 和 1520 的乘积, 1101 和 1010 的乘积, 1234 和 4321 的乘积, 输入为 2304, 1520, 1101, 1010, 1234 和 4321 输出结果为 3502080, 1112010, 5332114 结果如图所示:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-3.exe

```

x * y = 3502080
x * y = 3502080
x * y = 1112010
x * y = 1112010
x * y = 5332114
x * y = 5332114
请按任意键继续. . .

```

5.算法分析:

当 $n=1$ 时, 时间复杂度为 $O(1)$. 当 $n>1$ 时, $T(n) = 3T(n/2) + O(n)$

即, 时间复杂度: $O(n^{1.59})$

实验 2-4 棋盘覆盖问题

1.问题描述:

在一个 $2^k \times 2^k$ 个方格组成的棋盘中, 恰有一个方格与其它方格不同, 称为特殊方格或缺陷方格, 且称该棋盘为一特殊棋盘。在棋盘覆盖问题中, 要用图示的 4 种不同形态的 L 型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格, 且任何 2 个 L 型骨牌不得重叠覆盖。在 $2^k \times 2^k$ 的棋盘覆盖中, 用到的 L 型骨牌数恰为 $(4k-1)/3$, 即 (所有方格个数-特殊方格个数) / 3。

2.算法实例:

如图, 为 $k=2$ 时的一个特殊棋盘 (相同颜色的三个小方格组成一个 L 型骨牌) 和 4 种不同形态的 L 型骨牌, 蓝色的为特殊方格。



3.程序代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>

#include<iomanip>
//setw(n)是 c++中在输出操作中使用的字段宽度设置, n 表示字段宽度。
using namespace std;

int matrix[100][100];
int num = 0;
void chessBoard(int x, int y, int a, int b, int length);
```

```

int main()
{
    int a, b, length;
    cout << "请输入棋盘的行列号";
    cin >> length;
    cout << "请输入特殊方格的行列号";
    cin >> a >> b;
    matrix[a][b] = 0;

    chessBoard(1, 1, a, b, length);

    for (int i = 1; i <= length; i++){
        for (int j = 1; j <= length; j++){
            cout << setw(4) << matrix[i][j];
        }
        cout << endl;
    }

    system("PAUSE");

    return 0;
}

void chessBoard(int x, int y, int a, int b, int length) {
    //如果棋盘上只有一个方格，且该方格为一特殊方格
    if (length == 1){
        return;
    }
    int h = length / 2;    //分割棋盘
    // int t = ++num;        //L 型骨牌号(从 1 开始)
    int t = num++;        //L 型骨牌号(从 0 开始)
    /*左上角*/
    if (a < x + h && b < y + h){    //特殊方格在此棋盘中则划分
        chessBoard(x, y, a, b, h);
    }
    else{    //否则先覆盖左右下角的方格(用 t 号 L 型骨牌)，再划分
        matrix[x + h - 1][y + h - 1] = t;
        chessBoard(x, y, x + h - 1, y + h - 1, h);
    }
    /*右上角*/
    if (a < x + h && b >= y + h){    //特殊方格在此棋盘中则划分
        chessBoard(x, y+h, a, b, h);
    }
    else{    //否则先覆盖左下角的方格(用 t 号 L 型骨牌)，再划分
        matrix[x + h - 1][y + h] = t;
        chessBoard(x, y+h, x + h - 1, y + h, h);
    }
}

```

```

/*右下角*/
if (a >= x + h && b >= y + h){    //特殊方格在此棋盘中则划分
    chessBoard(x+h, y+h, a, b, h);
}
else{    //否则先覆盖左上角的方格(用 t 号 L 型骨牌)，再划分
    matrix[x + h][y + h] = t;
    chessBoard(x+h, y+h, x + h, y + h, h);
}

/*左下角*/
if (a >= x + h && b < y + h){    //特殊方格在此棋盘中则划分
    chessBoard(x+h, y, a, b, h);
}
else{    //否则先覆盖右上角的方格(用 t 号 L 型骨牌)，再划分
    matrix[x + h][y + h - 1] = t;
    chessBoard(x+h, y, x + h, y + h - 1, h);
}
}
}

```

4.程序验证:

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-4.exe

请输入棋盘的行列号8
请输入特殊方格的行列号5 2

验证 8*8 的棋盘下，缺陷位置为第五行第二列处，输入为 8，5，2，L 型骨牌填充次序，数字为次序。
输出结果如图所示：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\2-4.exe

请输入棋盘的行列号8
请输入特殊方格的行列号5 2

2	2	3	3	7	7	8	8
2	1	1	3	7	6	6	8
5	1	4	4	10	10	6	9
5	5	4	0	0	10	9	9
17	0	18	18	0	12	13	13
17	17	16	18	12	12	11	13
20	16	16	19	15	11	11	14
20	20	19	19	15	15	14	14

请按任意键继续. . .

5.算法分析:

给定 $2^k \times 2^k$ 棋盘,当 $k=0$ 时, $T(k) = O(1)$, 当 $k>0$ 时, $T(k) = 4T(k-1) + O(1)$
所以, 时间复杂度: $O(4^k)$

分析与总结：

分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。在我碰到的实际问题中，比如回溯法，往往需要剪枝，来提前终止递归，以此达到优化算法的目的。因为递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。所以普遍情况下需要去优化算法，剪枝便是其中比较常见的，也是我经常接触的一种方法。我学过的极大极小值剪枝法，这个应用在计算机博弈中。这些是我这次上级课学到的，也是我的一些学习算法的体会。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：